Boba System Database

Revision: 2 **Last modified:** 2026-08-21T15:02:00Z

The `boba-jackett` Go service (port 7189) owns a SQLite system database at `config/boba.db` (default; override with `BOBA_DB_PATH`). Sensitive columns are encrypted with `BOBA_MASTER_KEY` (AES-256-GCM).

This document is the canonical reference for the schema, the master key lifecycle, backup, file permissions, and recovery from corruption.

1. Schema reference

Defined in `qBitTorrent-go/internal/db/migrations/001_init.sql`. See the spec at `docs/superpowers/specs/2026-04-27-jackett-management-ui-and-system-db-design.md` Â§ 5 for full normative text.

Table	Purpose	Encrypted columns
`credentials`	Per-tracker credential bundle. `kind` is `userpass` or `cookie`. Holds `username_enc`, `password_enc`, `cookies_enc` (any combination per `kind`). Tracks `created_at`, `updated_at`, `last_used_at`.	`username_enc`, `password_enc`, `cookies_enc`
`indexers`	Local registry of indexers configured at Jackett. Each row points (optionally) at a `credentials` row by `linked_credential_name`. Tracks `enabled_for_search`, `last_jackett_sync_at`, `last_test_status`, `last_test_at`.	none
`catalog_cache`	Snapshot of Jackettâ€™s indexer catalog (last refresh). UI browse + filter source. `template_fields_json` is the per-indexer config template fetched from Jackett.	none
`autoconfig_runs`		

Table	Purpose	Encrypted columns
	Append-only history of every auto-config run. Stores `ran_at`, counts, redacted `result_summary_json`, `errors_json`. Indexed by `ran_at DESC` for the UI runs page.	none (summary is already redacted)
`indexer_map_overrides`	Explicit `env_name â†’ indexer_id` map. Highest precedence in the matcher (beats fuzzy matching).	none
`schema_migrations`	Schema version tracking; row `1` corresponds to `001_init.sql`.	none

Indexes

- `idx_indexers_linked_cred` on `indexers(linked_credential_name)`
- `idx_catalog_cached_at` on `catalog_cache(cached_at)`
- `idx_runs_ran_at` on `autoconfig_runs(ran_at DESC)`

Pragmas

The DB is opened with WAL journaling, foreign keys on, and busy-timeout sufficient for the bootstrap import + UI write rate.

2. Master key lifecycle

Auto-generation (canonical path)

`BOBA_MASTER_KEY` is a 32-byte hex AES-256-GCM key. It is generated **exactly once**, on first boot, by `internal/bootstrap.EnsureMasterKey` (see `qBitTorrent-go/internal/bootstrap/bootstrap.go`).

`start.sh` also calls an `ensure_boba_master_key` helper as a belt-and-suspenders so the variable is present in the hostâ€™s `.env` even if the container has not yet booted.

The two-step write (key generated in memory, then written to .env) was collapsed into a single atomic `envfile.Atomic` write at commit `c0f20bc` to eliminate a silent data-loss window. The regression guard is `TestEnsureMasterKeyDoesNotDuplicateHeader` in `qBitTorrent-go/internal/bootstrap/bootstrap_test.go`.

Key Rotation

> **STATUS (verified 2026-08-20): NOT EXECUTABLE AS WRITTEN.** Both subcommands below â€” `rotate-key` and `envfile-replace` â€” are **absent from the source**. Verified by a control-needle-checked search of `qBitTorrent-go/**/*.go` (Â§11.4.201(7)(b): the needle `EnsureMasterKey` matched, so the absences are real, not a blind grep). There is also no built `bin/boba-jackett` to probe. Treat the block below as the INTENDED design, not a working runbook: today, use the â€œLoss recoveryâ€ path with `.env` as the source of truth. Tracked as `TODO(BOB_MASTER_KEY_ROTATION)`.
>
> Heading note: this section is named **â€œKey Rotationâ€** because `internal/bootstrap/bootstrap.go` writes `# To rotate: see docs/BOBA_DATABASE.md Â§ "Key Rotation".` into every operatorâ€™s `.env`. It was previously titled â€œManual rotationâ€, so that shipped pointer resolved to nothing.

To rotate the master key without losing credentials (INTENDED design):

```
# 1. Stop the service (use the orchestrator, never raw podman/docker)
make boba-jackett-stop   # or your orchestrator's equivalent

# 2. Decrypt all rows with the OLD key, re-encrypt with a NEW key.
#    Use the rotate subcommand:
./bin/boba-jackett rotate-key \
  --db /config/boba.db \
  --old-key "$OLD_KEY" \
  --new-key "$NEW_KEY"

# 3. Atomically update .env (envfile.Atomic uses tmpfile + fsync + rename)
./bin/boba-jackett envfile-replace BOBA_MASTER_KEY "$NEW_KEY"

# 4. Start the service
make boba-jackett-start
```

(Implementation note: a `rotate-key` subcommand is the canonical place for this operation; if not yet present in the binary, follow the â€œloss recoveryâ€ path below using `.env` as the source of truth.)

Loss recovery

There is **no recovery** from `BOBA_MASTER_KEY` loss alone. Encrypted rows in `credentials` become permanently undecipherable.

The only recovery path: the original `<NAME>_USERNAME` / `<NAME>_PASSWORD` / `<NAME>_COOKIES` triples in `.env` (or wherever they were originally entered) are still present. To re-bootstrap:

```
# 1. Stop the service
# 2. Move the now-useless DB out of the way
```

```
mv config/boba.db config/boba.db.lost
# 3. Generate a new master key (or let bootstrap do it)
sed -i '/^BOBA_MASTER_KEY=/d' .env
# 4. Start the service â€” bootstrap.EnsureMasterKey will generate a fresh
#    key and re-import credentials from .env into a brand new boba.db.
make boba-jackett-start
```

UI-only edits made after the original bootstrap are NOT recoverable â€” they were never written back to `.env`.

3. Backup procedure

> **Performable as of 2026-08-21.** This procedure was previously impossible to carry out, and the document did not say so â€” it prescribed no workaround because it never recorded the blockage. `config/boba.db` is mode `600`, and its owner was a rootless-container sub-uid (`100999`) with no host account, so the operator could not read their own credential database and neither `cp` nor `sqlite3 .backup` could open it. Feature `002-user-owned-downloads` restored the file to the operator. **Measured 2026-08-21:**
>
> ```
> config/boba.db mode=600 owner=milosvasic(1000) group=milosvasic(1000)
> ```
>
> A copy of the file read cleanly (81920 bytes) and the source stayed mode `600` afterwards. No elevation, no `podman unshare`, and no chown-first step is needed â€” if you find yourself reaching for one, the ownership regression has returned: run `scripts/ownership_precondition.sh` (it names the offending location) and then `scripts/ownership_repair.sh`. See scripts/ownership_repair.md and guides/file-ownership.md.

Two acceptable approaches.

Cold backup (service stopped):

```
make boba-jackett-stop
cp config/boba.db config/boba.db.bak
cp .env .env.bak
make boba-jackett-start
```

Hot backup (service running): use SQLiteâ€™s online `.backup`:

```
sqlite3 config/boba.db ".backup config/boba.db.bak"
cp .env .env.bak
```

`.backup` respects WAL semantics and produces a transactionally consistent snapshot. **Always back up `boba.db.bak` and `.env.bak` together** â€” the DB is useless without the matching master key, and the master key is useless without the DB.

Both files MUST be stored with mode `0600` (or stricter) on persistent media; treat them as you would a private SSH key.

The fix restored the OWNER, not the mode â€” and the mode must stay `600`

`config/boba.db` remains mode `600` and **MUST** stay mode `600`. The ownership repair changes who owns the file; it explicitly does not relax who may read it (the file is declared `preserve_mode: true` in `config/owned_paths.yaml`, and `scripts/ownership_repair.sh` restores each itemâ€™s original permission bits after the `chown`, because a non-root `chown(2)` can clear them).

This distinction is the whole point. A backup that started succeeding because the file had become world-readable would not be a fix â€” it would be a security regression wearing a fixâ€™s clothes: a credential store made readable by every account on the host, traded for a convenience the correct owner already provides. The backup works now because the file **belongs** to the operator, not because it was opened up.

If `stat -c '%a %U' config/boba.db` ever reports anything wider than `600`, or an owner that is not the account running the system, stop and treat it as a defect. The standing guards are the Layer 4 challenge `boba_db_file_perms_challenge.sh` (Â§ 4 below) for the mode, and pre-build invariant 33 `CM-OWNERSHIP-INVARIANTS` for the ownership route.

4. File permissions

Both files MUST be `0600` (owner read/write only):

File	Enforced by	Code reference
`config/boba.db` (and `-wal`, `-shm` siblings)	`db.Open` chmods on first open if mode is wider	`qBitTorrent-go/internal/db/conn.go` line ~53
`.env`	`internal/envfile.Atomic` writes with mode `0600`	`qBitTorrent-go/internal/envfile/write.go`

The Layer 4 security challenge `boba_db_file_perms_challenge.sh` is the regression guard â€” it MUST PASS before any release. The corresponding Go test is `qBitTorrent-go/internal/db/credential_leak_test.go`.

The DB file mode bug (modernc.org/sqlite creating `boba.db` `0644`) was caught at commit `8405167` and fixed in `db.Open` with `os.Chmod(path, 0o600)`. See also `docs/issues/fixed/BUGFIXES.md`.

5. Recovery from a corrupted DB

If `boba.db` becomes corrupted (truncated, partial writes, foreign-key violations from manual edits, etc.), the recovery path is:

```
# 1. Stop the service (orchestrator)
make boba-jackett-stop

# 2. Move the broken DB aside (DO NOT delete â€” keep it for forensics)
mv config/boba.db config/boba.db.broken
# Also move WAL/SHM if present
mv config/boba.db-wal config/boba.db-wal.broken 2>/dev/null || true
mv config/boba.db-shm config/boba.db-shm.broken 2>/dev/null || true

# 3. Restart â€” bootstrap.EnsureMasterKey notices the missing DB,
#    re-creates the schema (migration 001), then re-imports any
#    credentials still present in .env.
make boba-jackett-start

# 4. Verify
curl -s http://localhost:7189/healthz | jq .
curl -s http://localhost:7189/api/v1/credentials | jq 'length'
```

Anything entered via the UI after the original bootstrap and before the corruption is unrecoverable from `.env` alone â€” see Â§ 2 â€œLoss recoveryâ€. Always pair regular backups (Â§ 3) with this recovery path.

6. Related files

- Schema source: `qBitTorrent-go/internal/db/migrations/001_init.sql`
- Open / chmod logic: `qBitTorrent-go/internal/db/conn.go`
- Master key bootstrap: `qBitTorrent-go/internal/bootstrap/bootstrap.go`
- Atomic .env writer: `qBitTorrent-go/internal/envfile/write.go`
- Credential repo: `qBitTorrent-go/internal/db/credentials.go`
- Bug history: `docs/issues/fixed/BUGFIXES.md`
- Integration overview: `docs/JACKETT_INTEGRATION.md` Â§ Auto-Configuration